# BDA Week 8-9 — Graph Mining: PageRank

Module 1 of Phase-1 Final Prep · Highest-yield post-mid topic

BDA Final Exam Prep · Comprehensive Tutor


**Week 08 – 09 · Graph Mining: PageRank**

- §1 — Beginning Key Notes (Study Compass)
- §2 — Why PageRank? The Web-Search Problem
- §3 — Flow Formulation: Links as Weighted Votes
  - The canonical 3-node example (y, a, m)
- §4 — Matrix Formulation: The Stochastic Adjacency Matrix M
- §5 — Eigenvector View & Power Iteration
  - Power Iteration Method
- §6 — Random Walk Interpretation & Stationary Distribution
- §7 — Two Convergence Problems: Dead-ends & Spider Traps
  - Problem A — Spider traps
  - Problem B — Dead-ends
- §8 — The Google Solution: Random Teleports
  - The teleport-augmented PageRank equation
- §9 — Sparse Matrix Formulation & The Complete Algorithm
  - Handling dead-ends without removing them
  - Algorithm — Complete PageRank (slide form)
  - Storage cost
- §10 — Eight Worked Numerical Examples
  - Worked Example 1 — Building M from a small graph
  - Worked Example 2 — Three power-iteration steps on y/a/m
  - Worked Example 3 — Spider trap demonstration
  - Worked Example 4 — Dead-end demonstration
  - Worked Example 5 — Google formulation with $\beta=0.8$
  - Worked Example 6 — Spider trap fixed by teleport ($\beta=0.8$)
  - Worked Example 7 — Dead-end fixed by re-injecting leaked mass
  - Worked Example 8 — 5-node graph with $\beta=0.85$
- §11 — Practice Questions (15)
- §12 — Full Worked Answers
- §13 — Ending Key Notes (Revision Cards)

## §14 — Formula & Algorithm Reference

# Week 08 – 09 · Graph Mining: PageRank

**Why this PDF first?** PageRank is the single highest-yield post-mid topic. Every BDA final paper this examiner has set asks at minimum a 15–20 mark numerical PageRank trace. Master §1–§9 here and you have ~25–35 marks locked in before walking into the hall.

## §1 — Beginning Key Notes (Study Compass)

These are the eight load-bearing ideas you must walk into the exam owning. Every numerical question on PageRank reduces to applying one of them.

1. **Links are votes.** A page is important if many important pages link to it. Self-referential definition → solve as an eigenvector problem.
2. **Flow equation:** $r_j = \sum_{i \to j} r_i / d_i$. Each in-link contributes the source page's rank divided by its out-degree.
3. **Matrix form:** $r = M \cdot r$ where $M$ is the column-stochastic transition matrix. $M_{ji} = 1/d_i$ if $i \to j$, else $0$.
4. **Power iteration:** $r^{(t+1)} = M \cdot r^{(t)}$, started from $r^{(0)} = [1/N, \ldots, 1/N]$, stop when $|r^{(t+1)} - r^{(t)}|_1 < \varepsilon$. Converges to the principal eigenvector with eigenvalue $1$.
5. **Two failure modes:** *Spider traps* absorb all rank, *dead-ends* leak rank out — both break naive PageRank.
6. **Google fix — random teleports:** with probability $\beta$ follow a link, with probability $1-\beta$ jump to a uniformly random page. $\beta \approx 0.8$–$0.9$.
7. **Google matrix:** $A = \beta M + \frac{1-\beta}{N} J_{N \times N}$. $A$ is stochastic, aperiodic, irreducible — unique stationary $r$ exists.
8. **Production form (sparse):** $r^{new} = \beta M r^{old} + [(1-S)/N]_N$, where $S = \sum_j (\beta M r^{old})_j$. The $(1-S)/N$ term re-injects dead-end leakage uniformly.

**The single biggest exam pattern.** The examiner consistently asks you to walk a small graph through 2–3 power-iteration steps by hand. Master that procedure and 60–70% of the PageRank marks are guaranteed.

## §2 — Why PageRank? The Web-Search Problem

The web is enormous, decentralized, and full of pages all claiming to be relevant. Two problems make ranking hard:

**(1) Trust.** Many pages publish information about the same topic. Who do you believe? Trustworthy pages tend to link to other trustworthy pages, so we can use the link graph itself to surface trust.

**(2) Relevance amid ambiguity.** If a user types *"newspaper"*, there is no single right answer. Pages that genuinely know about newspapers tend to link to many newspapers — again, structure in the link graph signals authority.

PageRank reframes the search-ranking problem as a question about graph structure: **given the link graph alone (forget the content for a moment), how important is each page?** The breakthrough idea — patented by Brin and Page in 1998 — is to define importance *recursively*: a page is important if important pages link to it.

> **Analogy — academic citations.** A paper cited 50 times by other heavily-cited papers is more important than one cited 50 times only by obscure papers. The citing paper's own importance flows into the cited paper. PageRank is exactly this idea applied to the web: a hyperlink is the web's version of a citation.

**Three guiding ideas before any maths:** - In-links are votes — but every vote is not equal. - A vote from an important page counts more than a vote from a low-importance page. - Each page's vote is split equally among its out-links — so a page with 1000 out-links passes only $1/10001/1000$ of its rank to each linked target.

---

## §3 — Flow Formulation: Links as Weighted Votes

Let $rjr_j$ denote the rank (importance score) of page $jj$. Let $did_i$ be the out-degree of page $ii$. If page $ii$ links to page $jj$, page $ii$ contributes $ri/dir_i$ / $d_i$ to page $jj$ — its own rank, divided evenly across its out-links. Summing over all in-links gives the central **flow equation**:

$rj=\sum i\to jridi$ r_j = \sum_{i \to j} \frac{r_i}{d_i}

This is a linear equation per page. With $NN$ pages we have $NN$ equations in $NN$ unknowns. But the equations are *homogeneous* — multiplying every $rjr_j$ by the same constant gives another valid solution. We close the system with a normalization constraint:

$\sum jrj=1$ \sum_j r_j = 1

### The canonical 3-node example (y, a, m)

The course slides (and most exam problems) use this graph:

- $y\to yy$ \to y (self-loop), $y\to ay$ \to a
- $a\to ya$ \to y, $a\to ma$ \to m
- $m\to am$ \to a

Out-degrees: $dy=2d_y = 2$, $da=2d_a = 2$, $dm=1d_m = 1$.

**Reading the graph into flow equations:**

$ry=ry2+ra2,ra=ry2+rm,rm=ra2$ r_y = \frac{r_y}{2} + \frac{r_a}{2}, \quad r_a = \frac{r_y}{2} + r_m, \quad r_m = \frac{r_a}{2}

Combined with $r_y + r_a + r_m = 1$, the unique solution is $r_y = 2/5$, $r_a = 2/5$, $r_m = 1/5$.

> **EXAM TRAP — out-degree, not in-degree.** In $r_i / d_i$, $d_i$ is the *source* page's out-degree, not the target's in-degree. Students routinely mix these up under exam pressure. Always label out-degrees on the graph FIRST, then write the flow equations.

## §4 — Matrix Formulation: The Stochastic Adjacency Matrix M

Define the **stochastic adjacency matrix** $M$ of size $N \times N$ as:

$$M_{ji} = \frac{1}{d_i} \;;\;\text{if } i \to j, \quad M_{ji} = 0 \;;\;\text{otherwise}$$

Read $M_{ji}$ as "column $i$, row $j$" — it is the share of $i$'s rank that flows to $j$. Each *column* of $M$ sums to $1$ (because page $i$ splits its outflow across exactly $d_i$ out-links, each getting $1/d_i$). That is what **column-stochastic** means.

> **Definition — column-stochastic matrix.** A non-negative matrix whose every column sums to $1$. Crucially, $M \cdot v$ preserves the L1 norm of $v$: if $\sum v_i = 1$ then $\sum (Mv)_j = 1$. This is why power iteration on $M$ does not blow up.

With $M$ defined, the flow equations bundle into a single matrix equation:

$$r = M \cdot r$$

**Building M for the y/a/m example.** Out-degrees: $d_y = 2$, $d_a = 2$, $d_m = 1$.

| from \ to | y | a | m |
|---|---|---|---|
| **y** | 1/2 | 1/2 | 0 |
| **a** | 1/2 | 0 | 1 |
| **m** | 0 | 1/2 | 0 |

Reading column-by-column: column y sends $\frac{1}{2}$ to y (self-loop) and $\frac{1}{2}$ to a. Column a sends $\frac{1}{2}$ to y and $\frac{1}{2}$ to m. Column m sends $1$ to a (its only out-link).

## §5 — Eigenvector View & Power Iteration

The equation $r = M \cdot r$ says exactly that $r$ is an **eigenvector of** $M$ **with eigenvalue** $1$. For a column-stochastic non-negative matrix $M$, the largest eigenvalue is always $1$, and the corresponding eigenvector — the principal eigenvector — is what we want.

> **Definition — eigenvector / eigenvalue.** A non-zero vector $x$ is an eigenvector of $A$ with eigenvalue $\lambda$ iff $A \cdot x = \lambda \cdot x$. $x$ captures a "direction" that $A$ only stretches/shrinks, never rotates. PageRank's key insight: $r$ is the eigenvector that $M$ leaves unchanged ($\lambda = 1$).

### Power Iteration Method

- Initialize $r(0) = [1/N, 1/N, \ldots, 1/N]^T$.
- Iterate: $r(t+1) = M \cdot r(t)$.
- Stop when $|r(t+1) - r(t)|_1 < \varepsilon$ (small threshold, e.g. $10^{-6}$).

**Why does this work?** Repeated multiplication by $M$ amplifies the principal eigen-direction relative to all others. After enough steps $r(t)$ aligns with the eigenvector of eigenvalue $1$, and applying $M$ one more time changes nothing — we have converged.

- Typical convergence: $\approx 50$ iterations to high precision on web-scale graphs.
- Each iteration is one sparse matrix–vector multiply: $O(N+E)$ operations where $E$ is the number of edges.

---

## §6 — Random Walk Interpretation & Stationary Distribution

There is a beautiful probabilistic reading of PageRank. Imagine a **random surfer** who, at each time step, picks one of the current page's out-links uniformly at random and follows it. Let $p_j(t)$ be the probability the surfer is at page $j$ at time $t$. Then:

$$p(t+1) = M \cdot p(t)$$

If the random walk reaches a state where $p(t+1) = p(t)$, that $p$ is a **stationary distribution** of the Markov chain. Comparing with $r = M \cdot r$ reveals the punchline:

> **PageRank is the stationary distribution of the random surfer.**

Pages with high PageRank are pages a random web-walker visits often.

**Existence and uniqueness** require the chain to be: - **Stochastic** — every column of $M$ sums to $1$ (no rank leakage). - **Aperiodic** — the chain is not stuck in a deterministic cycle. - **Irreducible** — every state is reachable from every other state.

Naive web graphs violate the first two conditions through dead-ends and spider traps. That is exactly the problem we tackle next.

---

## §7 — Two Convergence Problems: Dead-ends & Spider Traps

Run vanilla power iteration on the real web and one of two pathological behaviours appears.

### Problem A — Spider traps

A subset of pages whose out-links all point inside the subset. Once the random surfer enters, they cannot leave. Over enough iterations the entire rank mass concentrates inside the trap, and every page outside it ends with rank $00$.

**Example.** Modify the y/a/m graph so $mm$'s only out-link is to itself ($m \to mm$ $\to m$). Iterating $MM$ from $r(0)=(1/3,1/3,1/3)r^{(0)} = (1/3, 1/3, 1/3)$ produces:

| Iter | | | |
|---|---|---|---|
| 0 | 1/3 | 1/3 | 1/3 |
| 1 | 2/6 | 1/6 | 3/6 |
| 2 | 3/12 | 2/12 | 7/12 |
| 3 | 5/24 | 3/24 | 16/24 |
| ∞ | 0 | 0 | 1 |

### Problem B — Dead-ends

A page with no out-links at all. The corresponding column of $MM$ is entirely zero, $MM$ is no longer column-stochastic, and rank mass *leaks* out of the system. Eventually every rank goes to $00$.

> **Are these the same problem? No.** Spider traps don't violate column-stochastic $MM$; they trap the random walk in a sub-region. Dead-ends literally break column-stochastic $MM$ because mass is destroyed at the dead-end. **The same fix solves both**, but for different formal reasons — see §8.

---

## §8 — The Google Solution: Random Teleports

Brin and Page's elegant fix is to give the random surfer an **escape hatch**. At every step:

- with probability $\beta$, the surfer follows one of the current page's out-links uniformly at random;
- with probability $1-\beta$, the surfer **teleports** to a uniformly random page in the entire graph.

Common values $\beta \approx 0.8$–$0.9$ — meaning the surfer follows links most of the time but every $\approx 5$–$10$ steps takes a random jump. This single tweak fixes both pathologies at once.

### The teleport-augmented PageRank equation

$$r_j = \sum_{i \to j} \beta \frac{r_i}{d_i} + (1 - \beta) \frac{1}{N}$$

Equivalently, define the **Google matrix** $A$:

$$A = \beta M + \frac{1 - \beta}{N} J_{N \times N}$$

where $J_{N \times N}$ is the all-ones matrix. $A$ is column-stochastic, aperiodic, and irreducible — a unique stationary distribution $r$ exists, and power iteration $r^{(t+1)} = A \cdot r^{(t)}$ finds it.

> **Tax & redistribute.** Each iteration, tax every page a fraction $(1-\beta)$ of its current rank, pool the tax revenue, and redistribute it equally to all $N$ pages. The remaining $\beta$-fraction flows along the link structure as before. This view often makes hand-computation faster: compute $\beta M r$ first, sum, then add $(1-\beta)/N$ to every entry.

**How does this kill spider traps?** The teleport gives the surfer a finite probability to escape *every* step. Average time to escape $\approx 1/(1-\beta) \approx 5$ steps when $\beta = 0.8$. So mass cannot accumulate forever inside any sub-region.

**How does this kill dead-ends?** One option is to pre-process $M$ and remove dead-end nodes iteratively. The simpler production approach is to **handle leakage explicitly during iteration** — see §9.

---

## §9 — Sparse Matrix Formulation & The Complete Algorithm

On real web graphs $M$ has billions of rows but only $\approx 10$–$20$ non-zeros per column (average out-degree). Storing or multiplying the dense Google matrix $A$ is infeasible — but $A = \beta M + \frac{1-\beta}{N} J_{N \times N}$ can be applied without ever forming $A$:

$$r^{new} = \beta M r^{old} + \frac{1 - \beta}{N} \mathbf{1}_N$$

Step 1 multiplies the *sparse* $M$ with $r$ — cheap. Step 2 just adds a constant to every entry — also cheap.

### Handling dead-ends without removing them

If $M$ has dead-ends, $\beta M r^{old}$ sums to less than $1$: some mass $S$ leaked. Re-inject the leak uniformly over all $N$ pages:

$$r^{new}_j = (\beta M r^{old})_j + \frac{1 - S}{N}, \quad S = \sum_j (\beta M r^{old})_j$$

This guarantees $\sum r^{new} = 1$ every iteration even with dead-ends present — and is mathematically equivalent to the Google teleport formulation when there are no dead-ends.

### Algorithm — Complete PageRank (slide form)

**Input:** Directed graph $G$ (may contain spider traps and dead-ends), teleport parameter $\beta$.

**Output:** Rank vector $r^{new}$.

1. Initialize $r^{old}_j = 1/N$ for all $j$.
2. Repeat until $\sum_j |r^{new}_j - r^{old}_j| < \varepsilon$:
    1. For each $j$: $r'^{new}_j = \sum_{i \to j} \beta \cdot r^{old}_i / d_i$
    2. $S = \sum_j r'^{new}_j$
    3. For each $j$: $r^{new}_j = r'^{new}_j + (1 - S)/N$
    4. $r^{old} \leftarrow r^{new}$
3. Return $r^{new}$.

### Storage cost

Storing MM as a sparse list (page ii, out-degree did_i, list of out-neighbours) costs O(N+E)O(N + E) — about 40 bytes per page in 32-bit indexing. Two rank vectors roldr^{old}, rnewr^{new} cost O(N)O(N). Block-stripe schemes split rnewr^{new} across multiple blocks so only one block of the result vector is in memory at a time — see Topic-Sensitive PageRank in W12 for details.

---

## §10 — Eight Worked Numerical Examples

Every example below is the kind of multi-step trace your examiner expects you to write out. Read the worked solution once, then close this document and try to reproduce it from scratch on paper.

### Worked Example 1 — Building M from a small graph

**Problem.** Three pages A,B,CA, B, C with edges A→BA \to B, A→CA \to C, B→CB \to C, C→AC \to A, C→BC \to B. Write down the stochastic adjacency matrix MM.

**Step 1 — out-degrees.** dA=2d_A = 2 (A→B,A→CA \to B, A \to C). dB=1d_B = 1 (B→CB \to C). dC=2d_C = 2 (C→A,C→BC \to A, C \to B).

**Step 2 — fill M column by column.**

| | from A | from B | from C |
|---|---|---|---|
| row A | 0 | 0 | 1/2 |
| row B | 1/2 | 0 | 1/2 |
| row C | 1/2 | 1 | 0 |

**Step 3 — verify.** Each column sums to 11. M·rM \cdot r computes one round of rank flow.

### Worked Example 2 — Three power-iteration steps on y/a/m

**Problem.** Using MM from §4, perform three power iterations starting from r(0)=(1/3,1/3,1/3)r^{(0)} = (1/3, 1/3, 1/3).

Flow equations: ry=ry/2+ra/2r_y = r_y/2 + r_a/2, ra=ry/2+rmr_a = r_y/2 + r_m, rm=ra/2r_m = r_a/2.

**Iteration 1.** r(1)=M·r(0)r^{(1)} = M \cdot r^{(0)}: - ry(1)=(1/3)/2+(1/3)/2=2/6=1/3r_y^{(1)} = (1/3)/2 + (1/3)/2 = 2/6 = 1/3 - ra(1)=(1/3)/2+1/3=1/6+2/6=3/6=1/2r_a^{(1)} = (1/3)/2 + 1/3 = 1/6 + 2/6 = 3/6 = 1/2 - rm(1)=(1/3)/2=1/6r_m^{(1)} = (1/3)/2 = 1/6 - r(1)=(1/3,1/2,1/6)r^{(1)} = (1/3, 1/2, 1/6). Sum = 11 ✓

**Iteration 2.** r(2)=M·r(1)r^{(2)} = M \cdot r^{(1)}: - ry(2)=(1/3)/2+(1/2)/2=2/12+3/12=5/12r_y^{(2)} = (1/3)/2 + (1/2)/2 = 2/12 + 3/12 = 5/12 - ra(2)=(1/3)/2+1/6=2/12+2/12=4/12r_a^{(2)} = (1/3)/2 + 1/6 = 2/12 + 2/12 = 4/12 - rm(2)=(1/2)/2=3/12r_m^{(2)} = (1/2)/2 = 3/12 - r(2)=(5/12,4/12,3/12)r^{(2)} = (5/12, 4/12, 3/12). Sum = 11 ✓

**Iteration 3.** r(3)=M·r(2)r^{(3)} = M \cdot r^{(2)}: - ry(3)=(5/12)/2+(4/12)/2=9/24r_y^{(3)} = (5/12)/2 + (4/12)/2 = 9/24 - ra(3)=(5/12)/2+3/12=11/24r_a^{(3)} = (5/12)/2 + 3/12 = 11/24 -

rm(3)=(4/12)/2=4/24r_m^{(3)} = (4/12)/2 = 4/24 - r(3)=(9/24,11/24,4/24)r^{(3)} = (9/24, 11/24, 4/24). Sum = 11 ✓

**Limit.** Continuing, the sequence converges to (2/5,2/5,1/5)=(0.4,0.4,0.2)(2/5, 2/5, 1/5) = (0.4, 0.4, 0.2).

## Worked Example 3 — Spider trap demonstration

**Problem.** Modify mm to a self-loop. Show how rank flows.

New MM: column mm entries are (0,0,1)(0, 0, 1). Iterations:

| Iter | | | |
|---|---|---|---|
| 0 | 1/3 | 1/3 | 1/3 |
| 1 | 2/6 | 1/6 | 3/6 |
| 2 | 3/12 | 2/12 | 7/12 |
| 3 | 5/24 | 3/24 | 16/24 |

**Trend.** rmr_m grows: 1/3→1/2→7/12→2/3→...→11/3 \to 1/2 \to 7/12 \to 2/3 \to \ldots \to 1. yy and aa both decay to 00.

## Worked Example 4 — Dead-end demonstration

**Problem.** Make mm a dead-end (no out-links). Show that without leak handling, total rank decays.

MM has all-zero column mm:

| | from y | from a | from m |
|---|---|---|---|
| row y | 1/2 | 1/2 | 0 |
| row a | 1/2 | 0 | 0 |
| row m | 0 | 1/2 | 0 |

**Iter 1** from (1/3,1/3,1/3)(1/3, 1/3, 1/3): r=(2/6,1/6,1/6)r = (2/6, 1/6, 1/6). Sum =4/6≠1= 4/6 \neq 1. Mass leaked =2/6= 2/6.

**Iter 2** on un-renormalised: r=(3/12,2/12,1/12)r = (3/12, 2/12, 1/12). Sum =1/2= 1/2.

**Iter 3:** Sum ≈0.42\approx 0.42. Total rank halves every 2–3 iterations; after ∼40\sim 40 iterations everything is ≈0\approx 0.

## Worked Example 5 — Google formulation with β=0.8\beta = 0.8

**Problem.** Same MM as WE 2. Compute first iteration with the Google formulation.

**Step 1 — build A.** Aji=0.8·Mji+0.2/3=0.8·Mji+0.067A_{ji} = 0.8 \cdot M_{ji} + 0.2/3 = 0.8 \cdot M_{ji} + 0.067.

| A | from y | from a | from m |
|---|---|---|---|
| row y | 0.467 | 0.467 | 0.067 |
| row a | 0.467 | 0.067 | 0.867 |
| row m | 0.067 | 0.467 | 0.067 |

**Step 2 — A · r^{(0)}.** r(0)=(1/3,1/3,1/3)r^{(0)} = (1/3, 1/3, 1/3). - ry(1)≈0.334r_y^{(1)} \approx 0.334, ra(1)≈0.467r_a^{(1)} \approx 0.467, rm(1)≈0.200r_m^{(1)} \approx 0.200.

**Faster route — tax & redistribute.** Mr(0)=(1/3,1/2,1/6)M r^{(0)} = (1/3, 1/2, 1/6) (from WE 2). Multiply by 0.80.8: (0.267,0.400,0.133)(0.267, 0.400, 0.133). Add 0.0670.067 to each: (0.333,0.467,0.200)(0.333, 0.467, 0.200). ✓

## Worked Example 6 — Spider trap fixed by teleport (β=0.8\beta = 0.8)

**Problem.** Repeat WE 3 (spider trap) with the Google formulation.

**Iter 1.** Mr(0)=(2/6,1/6,3/6)=(0.333,0.167,0.500)M r^{(0)} = (2/6, 1/6, 3/6) = (0.333, 0.167, 0.500). βMr=(0.267,0.133,0.400)\beta M r = (0.267, 0.133, 0.400). Add 0.0670.067: r(1)=(0.333,0.200,0.467)r^{(1)} = (0.333, 0.200, 0.467).

**Iter 2.** Mr(1)=(0.267,0.167,0.567)M r^{(1)} = (0.267, 0.167, 0.567). βMr=(0.213,0.133,0.453)\beta M r = (0.213, 0.133, 0.453). Add 0.0670.067: r(2)=(0.280,0.200,0.520)r^{(2)} = (0.280, 0.200, 0.520).

**Iter 3.** r(3)≈(0.259,0.179,0.563)r^{(3)} \approx (0.259, 0.179, 0.563).

Converged: r≈(0.21,0.15,0.64)r \approx (0.21, 0.15, 0.64). mm is most important but rank doesn't collapse to (0,0,1)(0,0,1).

## Worked Example 7 — Dead-end fixed by re-injecting leaked mass

**Problem.** Repeat WE 4 with leak handling, β=0.8\beta = 0.8.

**Iter 1.** Mr(0)=(0.333,0.167,0.167)M r^{(0)} = (0.333, 0.167, 0.167). βMr=(0.267,0.133,0.133)\beta M r = (0.267, 0.133, 0.133). S=0.533S = 0.533. Add (1−0.533)/3=0.156(1 - 0.533)/3 = 0.156: r(1)=(0.422,0.289,0.289)r^{(1)} = (0.422, 0.289, 0.289). Sum =1.000= 1.000 ✓.

**Iter 2.** Mr(1)=(0.356,0.211,0.144)M r^{(1)} = (0.356, 0.211, 0.144). βMr=(0.284,0.169,0.116)\beta M r = (0.284, 0.169, 0.116). S=0.569S = 0.569. Add (1−0.569)/3=0.144(1 - 0.569)/3 = 0.144: r(2)=(0.428,0.313,0.260)r^{(2)} = (0.428, 0.313, 0.260). Sum ≈1\approx 1 ✓.

Converges to ≈(0.41,0.32,0.27)\approx (0.41, 0.32, 0.27) — non-zero everywhere, no leakage.

## Worked Example 8 — 5-node graph with β=0.85\beta = 0.85

**Problem.** Nodes {A,B,C,D,E}\{A, B, C, D, E\}, edges A→BA \to B, A→CA \to C, B→CB \to C, C→AC \to A, D→AD \to A, D→BD \to B, E→DE \to D. One iteration from uniform r(0)r^{(0)}.

**Out-degrees:** dA=2d_A = 2, dB=1d_B = 1, dC=1d_C = 1, dD=2d_D = 2, dE=1d_E = 1.

**M (column-stochastic):**

| | fr A | fr B | fr C | fr D | fr E |
|---|---|---|---|---|---|
| row A | 0 | 0 | 1 | 1/2 | 0 |
| row B | 1/2 | 0 | 0 | 1/2 | 0 |
| row C | 1/2 | 1 | 0 | 0 | 0 |
| row D | 0 | 0 | 0 | 0 | 1 |
| row E | 0 | 0 | 0 | 0 | 0 |

**Step 4 — M · r^{(0)}** with $r(0)=0.2r^{(0)} = 0.2$ each: - $rA'=1\cdot0.2+0.5\cdot0.2=0.300r'_A = 1 \cdot 0.2 + 0.5 \cdot 0.2 = 0.300$ - $rB'=0.5\cdot0.2+0.5\cdot0.2=0.200r'_B = 0.5 \cdot 0.2 + 0.5 \cdot 0.2 = 0.200$ - $rC'=0.5\cdot0.2+1\cdot0.2=0.300r'_C = 0.5 \cdot 0.2 + 1 \cdot 0.2 = 0.300$ - $rD'=1\cdot0.2=0.200r'_D = 1 \cdot 0.2 = 0.200$ - $rE'=0r'_E = 0$

**Step 5 — apply $\beta$\beta and add teleport.** $\beta Mr(0)=(0.255,0.170,0.255,0.170,0)\beta M r^{(0)} = (0.255, 0.170, 0.255, 0.170, 0)$. $(1-\beta)/N=0.030(1-\beta)/N = 0.030$.

$r(1)=(0.285,0.200,0.285,0.200,0.030)r^{(1)} = (0.285, 0.200, 0.285, 0.200, 0.030)$. Sum $=1.000= 1.000$ ✓

After one iteration $AA$ and $CC$ lead. $EE$ has no in-links so its rank equals the teleport mass.

## §11 — Practice Questions (15)

Mix: 6 numerical traces, 5 conceptual short-answer, 4 MCQ / true-false. Time yourself: $\sim6\sim 6$ min per numerical, $\sim3\sim 3$ min per conceptual, $\sim1\sim 1$ min per MCQ.

**Q1 [Numerical · 6 marks].** Three pages $\{A,B,C\}\{A, B, C\}$ with edges $A\to BA \to B$, $A\to CA \to C$, $B\to AB \to A$, $C\to AC \to A$, $C\to BC \to B$. (a) Write $MM$. (b) Verify column-stochastic. (c) From $r(0)=(1/3,1/3,1/3)r^{(0)} = (1/3, 1/3, 1/3)$, do two power-iteration steps without teleport.

**Q2 [Numerical · 8 marks].** Using the y/a/m graph and Google formulation with $\beta=0.8\beta = 0.8$, compute $r(1)r^{(1)}$ and $r(2)r^{(2)}$ from $r(0)=(1/3,1/3,1/3)r^{(0)} = (1/3, 1/3, 1/3)$. Use the tax & redistribute trick. Show all intermediate values.

**Q3 [Numerical · 7 marks].** Graph with $mm$ a dead-end: edges $y\to yy \to y$, $y\to ay \to a$, $a\to ya \to y$, $a\to ma \to m$. Run two iterations of the COMPLETE algorithm with leak re-injection, $\beta=0.85\beta = 0.85$, $r(0)r^{(0)}$ uniform. Compute $SS$ each step.

**Q4 [Numerical · 6 marks].** Graph $\{1,2,3,4\}\{1, 2, 3, 4\}$ with edges $1\to21 \to 2$, $2\to32 \to 3$, $3\to43 \to 4$, $4\to24 \to 2$. With $\beta=0.8\beta = 0.8$ and $r(0)r^{(0)}$ uniform, compute one iteration. Is $\{2,3,4\}\{2, 3, 4\}$ a spider trap?

**Q5 [Numerical · 5 marks].** $r(t)=(0.3,0.4,0.3)r^{(t)} = (0.3, 0.4, 0.3)$ for the y/a/m graph, $\beta=1\beta = 1$. Compute $r(t+1)r^{(t+1)}$. State $|r(t+1)-r(t)|1|r^{(t+1)} - r^{(t)}|_1$. With $\varepsilon=0.05\varepsilon = 0.05$, has the algorithm converged?

**Q6 [Numerical · 8 marks].** $M=(01/2001/201001/2011/2000)M = \begin{pmatrix} 0 & 1/2 & 0 & 0 \\ 1/2 & 0 & 1 & 0 \\ 0 & 1/2 & 0 & 1 \\ 1/2 & 0 & 0 & 0 \end{pmatrix}$ (cols = A, B, C, D). With $\beta=0.8\beta = 0.8$ and $r(0)=(1/4)\mathbf{1}r^{(0)} = (1/4)\mathbf{1}$, do one Google-formulation iteration.

**Q7 [Concept · 4 marks].** Explain in 4–6 sentences why $rr$ is the principal eigenvector of $MM$ with eigenvalue $11$. Why is the largest eigenvalue of a column-stochastic non-negative matrix necessarily $11$?

**Q8 [Concept · 4 marks].** Compare and contrast spider traps and dead-ends as failure modes. State (a) what happens to the rank vector with continued iteration, (b) which Markov-chain condition is violated, (c) how teleport / leak injection fixes it.

**Q9 [Concept · 3 marks].** Why $\beta\beta$ between $0.80.8$ and $0.90.9$? Trade-off as $\beta\to 1\beta \to 1$? As $\beta\to 0\beta \to 0$?

**Q10 [Concept · 4 marks].** State the random-walk interpretation of PageRank. Why is the stationary distribution of the modified chain (with teleport) unique while the original may not be?

**Q11 [Concept · 3 marks].** Show why $r=\beta Mr+(1-\beta)/N\cdot\mathbf{1}Nr = \beta M r + (1-\beta)/N \cdot \mathbf{1}_N$ is equivalent to $r=A\cdot rr = A \cdot r$ when $MM$ has no dead-ends. Why must we add a leak correction term when $MM$ does have dead-ends?

**Q12 [MCQ · 2 marks].** Which is TRUE for column-stochastic $MM$? (A) every row sums to 1 (B) every column sums to 1 and the largest eigenvalue is exactly 1 (C) $MrM r$ is always a probability distribution if $rr$ is, regardless of the graph (D) all entries of $MM$ are positive

**Q13 [MCQ · 2 marks].** Power iteration on column-stochastic $MM$ without traps/dead-ends converges to: (A) the principal eigenvector with eigenvalue 1 (B) any eigenvector (C) the zero vector (D) a vector that depends on $r(0)r^{(0)}$

**Q14 [True/False · 2 marks].** Increasing $\beta\beta$ from $0.80.8$ to $0.950.95$ makes power iteration converge faster. Justify.

**Q15 [MCQ · 2 marks].** If leaked mass $S=0.92S = 0.92$, $\beta=0.85\beta = 0.85$, $N=100N = 100$, the per-page leak correction is closest to: (A) 0.0008 (B) 0.0015 (C) 0.0085 (D) 0.0500

---

## §12 — Full Worked Answers

**A1.** (a) $dA=2d_A = 2$, $dB=1d_B = 1$, $dC=2d_C = 2$. $M=(011/21/201/21/200)M = \begin{pmatrix} 0 & 1 & 1/2 \\ 1/2 & 0 & 1/2 \\ 1/2 & 0 & 0 \end{pmatrix}$ (cols = A, B, C). (b) Column sums: $0+1/2+1/2=10 + 1/2 + 1/2 = 1$, $1+0+0=11 + 0 + 0 = 1$, $1/2+1/2+0=11/2 + 1/2 + 0 = 1$. ✓ (c) **Iter 1:** $rA=0\cdot 1/3+1\cdot 1/3+1/2\cdot 1/3=1/2r_A = 0 \cdot 1/3 + 1 \cdot 1/3 + 1/2 \cdot 1/3 = 1/2$; $rB=1/2\cdot 1/3+0+1/2\cdot 1/3=1/3r_B = 1/2 \cdot 1/3 + 0 + 1/2 \cdot 1/3 = 1/3$; $rC=1/2\cdot 1/3=1/6r_C = 1/2 \cdot 1/3 = 1/6$. $r(1)=(1/2,1/3,1/6)r^{(1)} = (1/2, 1/3, 1/6)$. **Iter 2:** $rA=0+1/3+1/12=5/12r_A = 0 + 1/3 + 1/12 = 5/12$; $rB=1/4+1/12=4/12r_B = 1/4 + 1/12 = 4/12$; $rC=1/4=3/12r_C = 1/4 = 3/12$. $r(2)=(5/12,4/12,3/12)r^{(2)} = (5/12, 4/12, 3/12)$.

**A2.** $Mr(0)=(1/3,1/2,1/6)\approx(0.333,0.500,0.167)M r^{(0)} = (1/3, 1/2, 1/6) \approx (0.333, 0.500, 0.167)$.

**Iter 1.** $\beta Mr=0.8\cdot(0.333,0.500,0.167)=(0.267,0.400,0.133)\beta M r = 0.8 \cdot (0.333, 0.500, 0.167) = (0.267, 0.400, 0.133)$. Add $0.0670.067$: $r(1)=(0.333,0.467,0.200)r^{(1)} = (0.333, 0.467, 0.200)$.

**Iter 2.** Mr(1)M r^{(1)}: ry′=0.333/2+0.467/2=0.400r_y' = 0.333/2 + 0.467/2 = 0.400; ra′=0.333/2+0.200=0.367r_a' = 0.333/2 + 0.200 = 0.367; rm′=0.467/2=0.233r_m' = 0.467/2 = 0.233. βMr=(0.320,0.293,0.187)\beta M r = (0.320, 0.293, 0.187). Add 0.0670.067: r(2)=(0.387,0.360,0.253)r^{(2)} = (0.387, 0.360, 0.253).

**A3.** M=(0.50.500.50000.50)M = \begin{pmatrix} 0.5 & 0.5 & 0 \\ 0.5 & 0 & 0 \\ 0 & 0.5 & 0 \end{pmatrix} (column mm is zero).

**Iter 1.** Mr(0)=(0.333,0.167,0.167)M r^{(0)} = (0.333, 0.167, 0.167). βMr=0.85·(…)=(0.283,0.142,0.142)\beta M r = 0.85 \cdot (\ldots) = (0.283, 0.142, 0.142). S=0.567S = 0.567. (1−S)/N=0.144(1-S)/N = 0.144. r(1)=(0.427,0.286,0.286)r^{(1)} = (0.427, 0.286, 0.286).

**Iter 2.** Mr(1)M r^{(1)}: ry′=0.5(0.427)+0.5(0.286)=0.357r_y' = 0.5(0.427) + 0.5(0.286) = 0.357; ra′=0.5(0.427)=0.214r_a' = 0.5(0.427) = 0.214; rm′=0.5(0.286)=0.143r_m' = 0.5(0.286) = 0.143. βMr(1)=(0.303,0.182,0.121)\beta M r^{(1)} = (0.303, 0.182, 0.121). S=0.606S = 0.606. (1−S)/N=0.131(1-S)/N = 0.131. r(2)=(0.434,0.313,0.252)r^{(2)} = (0.434, 0.313, 0.252).

**A4.** d1=d2=d3=d4=1d_1 = d_2 = d_3 = d_4 = 1. Mr(0)=(0,0.50,0.25,0.25)M r^{(0)} = (0, 0.50, 0.25, 0.25). βMr=(0,0.40,0.20,0.20)\beta M r = (0, 0.40, 0.20, 0.20). (1−β)/N=0.05(1-\beta)/N = 0.05. r(1)=(0.05,0.45,0.25,0.25)r^{(1)} = (0.05, 0.45, 0.25, 0.25).

**Spider trap?** Yes — once entered from node 1, the cycle {2,3,4}\{2, 3, 4\} traps the surfer. With β=0.8\beta = 0.8, teleport pulls 0.050.05 back to node 1 every step.

**A5.** MrM r: ry=0.3·0.5+0.4·0.5=0.35r_y = 0.3 \cdot 0.5 + 0.4 \cdot 0.5 = 0.35; ra=0.3·0.5+0.3=0.45r_a = 0.3 \cdot 0.5 + 0.3 = 0.45; rm=0.4·0.5=0.20r_m = 0.4 \cdot 0.5 = 0.20. r(t+1)=(0.35,0.45,0.20)r^{(t+1)} = (0.35, 0.45, 0.20). |r(t+1)−r(t)|1=0.05+0.05+0.10=0.20|r^{(t+1)} - r^{(t)}|_1 = 0.05 + 0.05 + 0.10 = 0.20. 0.20>0.050.20 > 0.05 → NOT converged.

**A6. Step 1:** Mr(0)M r^{(0)} with r(0)=(0.25)r^{(0)} = (0.25) each: - rA′=0+0.5·0.25+0+0=0.125r_A' = 0 + 0.5 \cdot 0.25 + 0 + 0 = 0.125 - rB′=0.5·0.25+0+1·0.25+0=0.375r_B' = 0.5 \cdot 0.25 + 0 + 1 \cdot 0.25 + 0 = 0.375 - rC′=0+0.5·0.25+0+1·0.25=0.375r_C' = 0 + 0.5 \cdot 0.25 + 0 + 1 \cdot 0.25 = 0.375 - rD′=0.5·0.25+0+0+0=0.125r_D' = 0.5 \cdot 0.25 + 0 + 0 + 0 = 0.125

Mr(0)=(0.125,0.375,0.375,0.125)M r^{(0)} = (0.125, 0.375, 0.375, 0.125). Sum =1= 1.

**Step 2:** βMr=(0.100,0.300,0.300,0.100)\beta M r = (0.100, 0.300, 0.300, 0.100).

**Step 3:** Add (1−β)/N=0.05(1-\beta)/N = 0.05 to each: r(1)=(0.150,0.350,0.350,0.150)r^{(1)} = (0.150, 0.350, 0.350, 0.150).

**A7.** r=Mrr = M r is the eigenvalue equation Ax=λxA x = \lambda x with A=MA = M, x=rx = r, λ=1\lambda = 1. So rr is an eigenvector of MM with eigenvalue 11. The all-ones row vector multiplied by MM equals the row of column sums, which is all-ones — meaning the all-ones vector is a LEFT eigenvector with eigenvalue 11. By Perron–Frobenius (for non-negative matrices), the largest eigenvalue equals the spectral radius. Combined with the row argument, λmax=1\lambda_{\max} = 1.

**A8. Spider trap.** (a) Rank inside the trap grows monotonically; eventually all rank concentrates there. (b) Markov chain becomes *reducible* — once entered, you cannot reach states outside. Aperiodicity may also fail. (c) Teleport gives probability 1−β1-\beta to escape every step, restoring irreducibility. **Dead-end.** (a) Total rank decays — every entry →0\to 0. (b) MM is not column-stochastic (zero column), so MrM r

doesn't preserve L1 norm. (c) Either remove dead-ends from $M$ or re-inject leaked mass $(1-S)/N$.

**A9.** $\beta$ balances **fidelity to link structure** (high $\beta$) vs **convergence speed and trap escape** (low $\beta$). $\beta \to 1$: vanilla iteration; traps and dead-ends dominate; convergence slow; rank may not be unique. $\beta \to 0$: rank approaches uniform; link structure ignored. $\beta = 0.85$ is empirically optimal.

**A10.** Random surfer picks an out-link uniformly at every page. PageRank $r_j$ = long-run fraction of time the surfer is at $j$ — the stationary distribution of the chain whose transition matrix is $M$. Without teleport, the chain may be reducible (traps) or sub-stochastic (dead-ends). With teleport, the modified chain is stochastic, irreducible, aperiodic — by the fundamental theorem of Markov chains, a unique stationary distribution exists.

**A11.** No dead-ends: $Ar = \beta Mr + (1-\beta)/N \cdot Jr = \beta Mr + (1-\beta)/N \cdot \mathbf{1}_N$ (since $\sum r_i = 1$, $Jr = \mathbf{1}_N$). ✓ With dead-ends, $Mr$ sums to $S < 1$. The simple form would only redistribute $(1-\beta)$ mass when in fact $(1-\beta) + (1-S)\beta$ mass needs redistributing. Replace $(1-\beta)/N$ with $(1-S)/N$ where $S = \beta \sum (Mr)$.

**A12. (B)**. (A) wrong (column-stochastic = columns sum to 1, not rows). (C) wrong with dead-ends. (D) wrong (only non-negative, not strictly positive).

**A13. (A)**. (B) wrong (power iteration picks the dominant eigenvector). (C) wrong (column-stochastic preserves sum 1). (D) wrong (irreducible aperiodic chain has unique limit).

**A14. False.** The convergence rate is governed by $|\lambda_2| \approx \beta$. Higher $\beta \to$ larger $\lambda_2 \to$ SLOWER convergence. $\beta = 0.85$ is a balance.

**A15.** Per-page correction $= (1-S)/N = 0.08/100 = 0.0008$. **(A)**. Independent of $\beta$ once $S$ is known.

# §13 — Ending Key Notes (Revision Cards)

| Term | Quick-fact |
|---|---|
| Out-degree did_i | Number of out-links from page ii. Denominator in flow equation. |
| Flow equation | rj=∑i→jri/dir_j = \sum_{i \to j} r_i / d_i. |
| Stochastic matrix MM | Mji=1/diM_{ji} = 1/d_i if i→ji \to j else 00. Each *column* sums to 1. |
| Matrix form | r=Mrr = M r. rr is the eigenvector of MM with eigenvalue 1. |
| Power iteration | r(t+1)=Mr(t)r^{(t+1)} = M r^{(t)}, start uniform, $$50 iters to converge. |
| Random walk view | rjr_j = stationary probability of a random surfer at page jj. |
| Stationary conditions | (i) stochastic, (ii) aperiodic, (iii) irreducible. |
| Spider trap | Set whose all out-links stay inside. Rank concentrates there. |
| Dead-end | Page with no out-links. Column of MM is zero → mass leaks. |
| Teleport (Google fix) | Prob. 1−β1-\beta, jump random. β≈0.8\beta \approx 0.8–0.90.9. Fixes both. |
| Google matrix AA | A=βM+(1−β)/N·JA = \beta M + (1-\beta)/N \cdot J. Stochastic, aperiodic, irreducible. |
| Tax & redistribute | rnew=βMrold+(1−β)/N·**1**Nr^{new} = \beta M r^{old} + (1-\beta)/N \cdot \mathbf{1} _N (no dead-ends). |
| Leak handling | S=∑(βMrold)S = \sum (\beta M r^{old}); add (1−S)/N(1-S)/N to every entry. |
| Convergence test | \|r(t+1)−r(t)\|1<ε\|r^{(t+1)} - r^{(t)}\|_1 < \varepsilon (ε≈10−6\varepsilon \approx 10^{-6}). |
| Mistake — out vs in | did_i is SOURCE ii's OUT-degree, not target's in-degree. |
| Mistake — col vs row | MM is column-stochastic. MjiM_{ji} = column ii, row jj. |
| Eigenvalue intuition | Largest eigenvalue of column-stochastic MM is exactly 1. |
| β\beta semantics | Expected link-follows before teleport. β=0.85\beta = 0.85 → ∼6.7\sim 6.7 link clicks. |

# §14 — Formula & Algorithm Reference

| Concept | Formula | When to use |
|---|---|---|
| Flow equation | $r_j = \sum_{i \to j} r_i / d_i$ | Defining PageRank from the link graph. |
| Matrix form | $r = M r$, $M_{ji} = 1/d_i$ if $i \to j$ else 0 | Compact statement; basis of all algorithms. |
| Eigenvector view | $r$ = principal eigenvector of $M$ with eigenvalue 1 | Justifying convergence of power iteration. |
| Power iteration | $r^{(t+1)} = M r^{(t)}$, $r^{(0)} = 1/N$ | Practical computation when $M$ is well-behaved. |
| Random walk | $r$ = stationary distribution of Markov chain $M$ | Probabilistic interpretation. |
| Google formulation | $r = \beta M r + (1-\beta)/N \cdot \mathbf{1}_N$ | Default formula with traps but no dead-ends. |
| Google matrix | $A = \beta M + (1-\beta)/N \cdot J$ | Theoretical statement of teleport fix. |
| Complete (with leak) | $r^{new}_j = \beta (M r^{old})_j + (1-S)/N$, $S = \sum \beta M r^{old}$ | Production form; handles dead-ends. |
| Convergence threshold | $\lvert r^{new} - r^{old} \rvert_1 < \varepsilon$ | Practical halting criterion. |

**Algorithmic complexity:** - One power-iteration on sparse $M$: $O(N+E)$. - Sparse $M$ storage: $\sim 10$ bytes per edge. - Typical iterations to convergence ($\beta = 0.85$, $\varepsilon = 10^{-6}$): 50–100. - Block-stripe matrix update: $O(N+E)$ work, $\sim N/k$ memory.

**Connections to other weeks:** - **W12 — Topic-Sensitive PageRank, TrustRank, HITS:** PageRank variants with biased teleport. - **W16 — Community Detection:** spectral methods use the same eigenvector machinery applied to the graph Laplacian. - **MapReduce iteration (MMDS §5.2.2):** matrix–vector multiply distributes across mappers/reducers.

---

*End of W08-09 PageRank exam-prep document.*